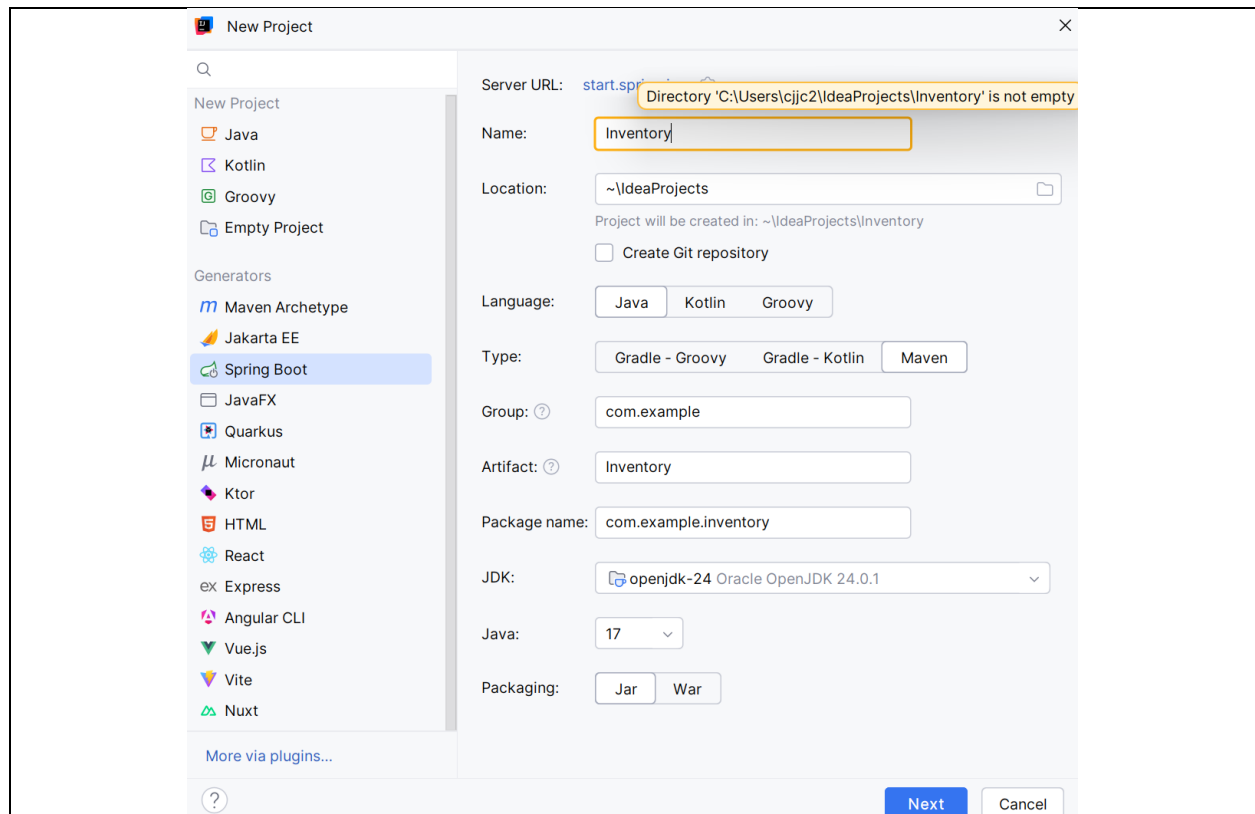
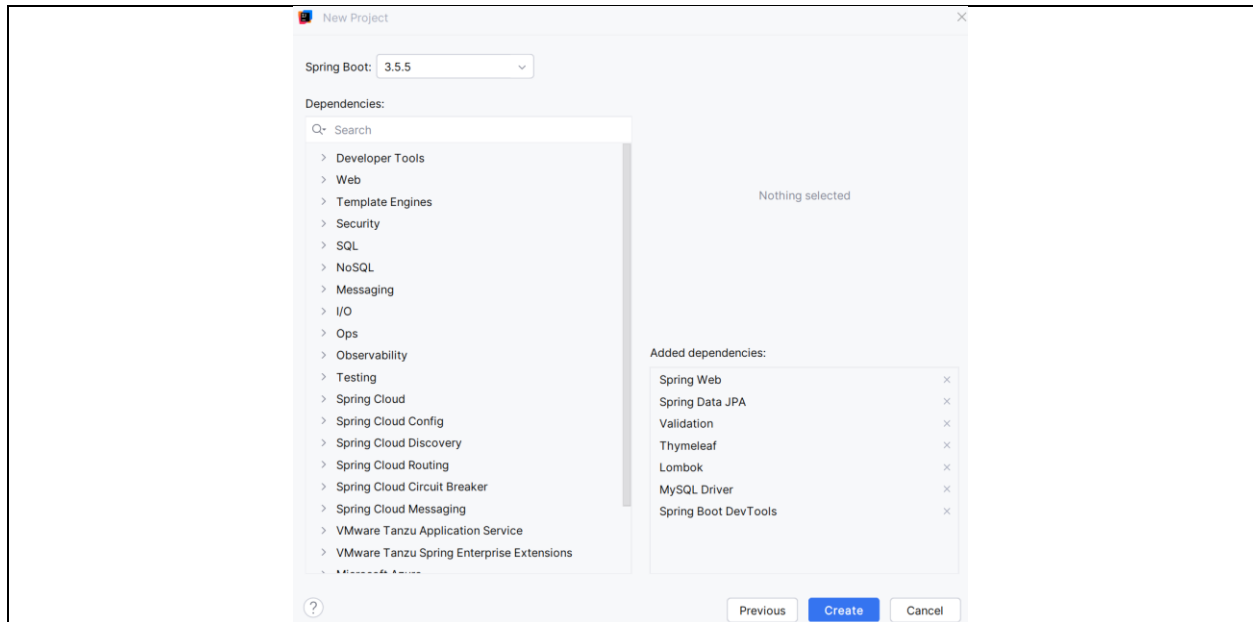


Lab 4- Java 17+, Spring Boot 3.x, IntelliJ, XAMPP/MySQL (running), Maven (Inventory)

1. Open XAMPP and run MySQL server
2. Open IntelliJ and create a new project Named the project student. Make sure you choose Maven and Jar.



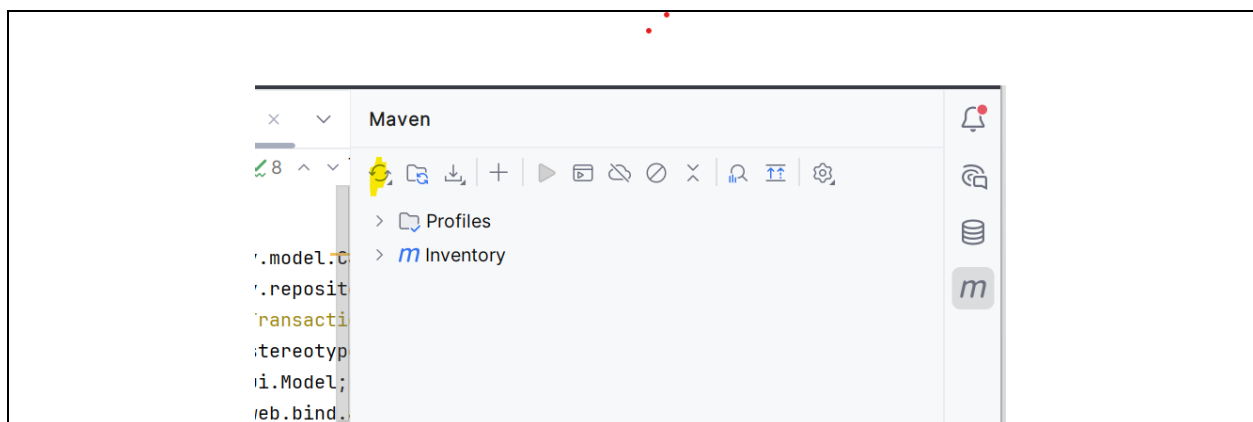
3. Add the dependencies needed for this project.



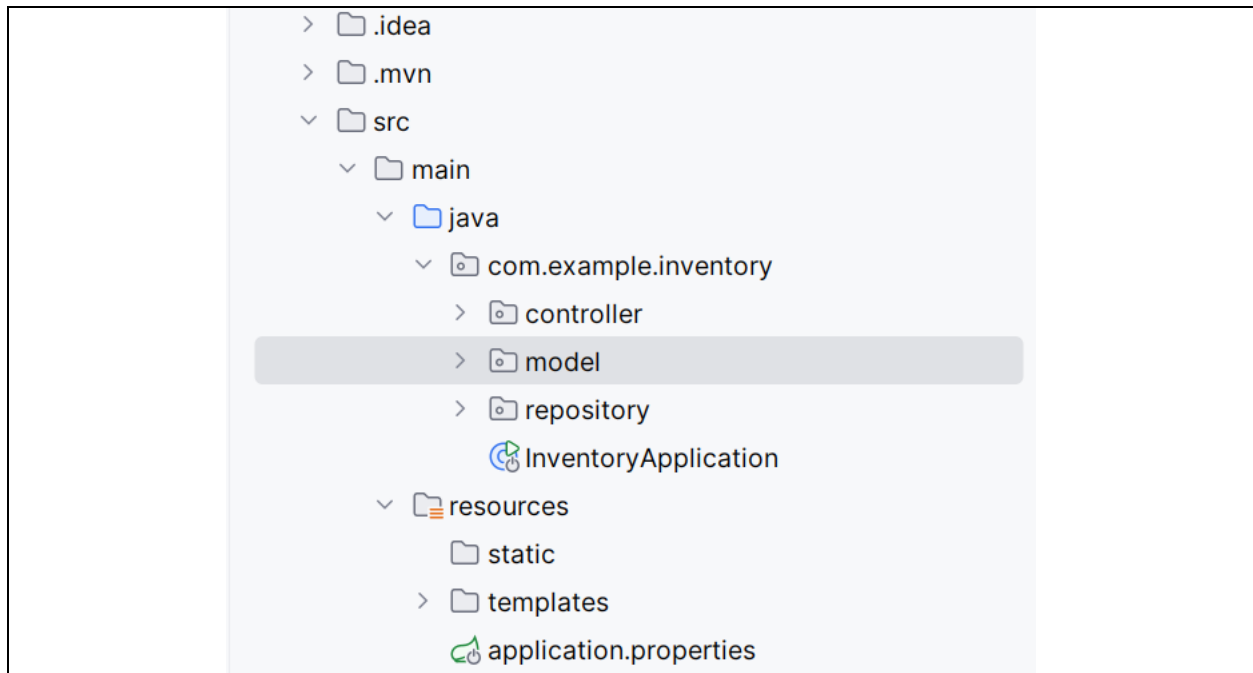
- Once you are inside the project, check your POM file and look at your dependencies. Check if you have spring devtools if not add it.



- Do not forget to sync your maven again after adding a new dependency.



6. Create the following folders in your directory. To create this, you need to create a package.



7. Set up your project in the application properties.

```
1  # --- Server settings ---
2  server.port=8080
3
4  # --- Database connection (MySQL) ---
5  spring.datasource.url=jdbc:mysql://localhost:3306/inventory?useSSL=false&serverTimezone=UTC
6  spring.datasource.username=root
7  spring.datasource.password=
8
9  # --- JPA / Hibernate ---
10 # Don't let Hibernate create/drop tables; just map to your existing DB
11 spring.jpa.hibernate.ddl-auto=none
12 spring.jpa.show-sql=true
13 spring.jpa.properties.hibernate.format_sql=true
14
15 # Prevents keeping DB sessions open while rendering Thymeleaf
16 spring.jpa.open-in-view=false
17
```

8. Create your POJO files. Inside model folder, create the following classes.

Category Class and Item Class

```
4
5 import jakarta.persistence.*;
6 import lombok.Getter;
7 import lombok.Setter;
8
9 import java.util.List;
10
11 @Entity
12 @Table(name = "category")
13 @Getter @Setter
14 public class Category {
15
16     @Id
17     private String catcode;
18
19     private String catdesc;
20
21     // One category has many items
22     @OneToMany(mappedBy = "category", fetch = FetchType.LAZY, cascade = CascadeType.ALL)
23     private List<Item> items;
24 }
25
```

```

import jakarta.persistence.*;
    import lombok.Getter;
import lombok.Setter;

@Entity
@Table(name = "items")
@Getter @Setter
public class Item {

    @Id
    private String itemcode;

    private String itemdesc;

    // Many items belong to one category
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "catcode", nullable = false)
    private Category category;
}

```

Note: The relationship between the two is established by mapping one class to another.

FetchType.Lazy means the actual objects won't be fetched from the database until you access it (for performance reasons).

9. Create the repository class and put it inside the repository folder.

```

import com.example.inventory.model.Category;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Modifying;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

import java.util.List;
import java.util.Optional;

public interface CategoryRepository extends JpaRepository<Category, String> { 3 usages
    // --- Simple reads (JPQL) ---
    @Query("select c from Category c order by c.catcode") 1 usage
    List<Category> findAllOrdered();
    // Load a category together with its items (relation) in one query:
    @Query("select c from Category c left join fetch c.items where c.catcode = :code") 1 usage
    Optional<Category> findWithItems(@Param("code") String code);
    // --- Native SQL examples
    // Search by description (native SQL)
    @Query(value = "SELECT * FROM category WHERE catdesc LIKE %:kw% ORDER BY catcode", nativeQuery = true) no usages
    List<Category> searchByDescription(@Param("kw") String keyword);
    // CategoryRepository.java
    @Query(value = "SELECT COUNT(*) FROM items WHERE catcode = :code", nativeQuery = true) 1 usage
    int countItems(@Param("code") String code);
    // Update (native SQL)
    @Modifying 1 usage
    @Query(value = "UPDATE category SET catcode=:newCode, catdesc=:newDesc WHERE catcode=:oldCode", nativeQuery = true)
    int updateCategory(@Param("oldCode") String oldCode,
        @Param("newCode") String newCode,
        @Param("newDesc") String newDesc);
}

```

10. Create the first page of your webapp. This page should be inside the templates folder.
Name this page category

```

<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Category List</title>
    <link rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/css/bootstrap.min.css">
</head>
<body class="container">

<h1>Item Categories</h1>

<a class="btn btn-success" href="/add-todo">Add</a>
<p class="text-success" th:if="${successMessage}"
th:text="${successMessage}"></p>
<p class="text-danger" th:if="${errorMessage}"
th:text="${errorMessage}"></p>

<table class="table table-striped">
    <thead>
    <tr>
        <th>Code</th>

```

```

        <th>Description</th>
        <th colspan="3">Actions</th>
    </tr>
</thead>
<tbody>
<tr th:each="c : ${categorylist}">
    <td th:text="${c.catcode}"></td>
    <td th:text="${c.catdesc}"></td>
    <td><a class="btn btn-primary" th:href="@{/update-
todo(id=${c.catcode})}">Edit</a></td>
    <td><a class="btn btn-danger"
        th:href="@{/delete-todo(id=${c.catcode})}"
        onclick="return confirm('Delete category ' +
/*[[${c.catcode}]]*/ ' + '? This cannot be undone.');">
        Delete
    </a>
    </td>
    <td><a class="btn btn-info" th:href="@{/see-
todo(id=${c.catcode})}">Items</a></td>
</tr>
</tbody>
</table>

</body>
</html>

```

11. Create the second page. Named this page items.

```

<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Category List</title>
    <link rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/css/bootstrap.mi
n.css">
</head>
<body class="container">

<h1>Item Categories</h1>

<a class="btn btn-success" href="/add-todo">Add</a>
<p class="text-success" th:if="${successMessage}"
th:text="${successMessage}"></p>
<p class="text-danger" th:if="${errorMessage}"
th:text="${errorMessage}"></p>

<table class="table table-striped">
    <thead>
        <tr>
            <th>Code</th>
            <th>Description</th>
            <th colspan="3">Actions</th>

```

```

</tr>
</thead>
<tbody>
<tr th:each="c : ${categorylist}">
  <td th:text="${c.catcode}"></td>
  <td th:text="${c.catdesc}"></td>
  <td><a class="btn btn-primary" th:href="@{/update-
todo(id=${c.catcode})}">Edit</a></td>
  <td><a class="btn btn-danger"
    th:href="@{/delete-todo(id=${c.catcode})}"
    onclick="return confirm('Delete category ' +
/*[[${c.catcode}]]*/ ' + '? This cannot be undone.');">
    Delete
  </a>
</td>
  <td><a class="btn btn-info" th:href="@{/see-
todo(id=${c.catcode})}">Items</a></td>
</tr>
</tbody>
</table>

</body>
</html>

```

12. Create the third page. Name it catser.

```

<!DOCTYPE html>
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8">
  <title>Add/Edit Category</title>
  <link rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/css/bootstrap.mi
n.css">
</head>
<body class="container">

<h1>Add or Edit Category</h1>

<form method="POST">
  <div class="form-group">
    <label for="ccode">Category Code</label>
    <input id="ccode" name="catcode" class="form-control"
      required th:attr="value=${id}">
  </div>
  <div class="form-group">
    <label for="cdesc">Category Description</label>
    <input id="cdesc" name="catdesc" class="form-control"
      required th:attr="value=${desc}">
  </div>
  <button type="submit" class="btn btn-success">Submit</button>

```

```
        <a th:href="@{/category}" class="btn btn-default" style="margin-left: 10px;">Cancel</a>
    </form>

</body>
</html>
```

13. Now, it is time to create your controller. Inside the controller folder create the controller class.

```
package com.example.inventory.controller;

import com.example.inventory.model.Category;
import com.example.inventory.repository.CategoryRepository;
import jakarta.transaction.Transactional;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.bind.support.SessionStatus;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;

@Controller
//@SessionAttributes({"errorMessage", "id"})
public class CategoryController {

    private final CategoryRepository categoryRepo;

    public CategoryController(CategoryRepository categoryRepo) {
        this.categoryRepo = categoryRepo;
    }

    @GetMapping("/{", "/category"})
    public String list(Model model) {
        model.addAttribute("categorylist",
            categoryRepo.findAllOrdered());
        return "category";
    }

    @GetMapping("/add-todo")
    public String showAdd(){ return "catser"; }

    @PostMapping("/add-todo")
    public String add(@RequestParam String catcode,
                     @RequestParam String catdesc,
                     RedirectAttributes ra) {
        if (categoryRepo.existsById(catcode)) {
            ra.addFlashAttribute("errorMessage", "Category code
already exists.");
            return "redirect:/category";
        }
        Category c = new Category();
        c.setCatcode(catcode);
    }
}
```

```

        c.setCatdesc(catdesc);
        categoryRepo.save(c);
        ra.addFlashAttribute("successMessage", "Category added.");
        return "redirect:/category";
    }

    @GetMapping("/update-todo")
    public String showEdit(@RequestParam("id") String id, Model model)
    {
        var c = categoryRepo.findById(id).orElse(null);
        if (c == null) {
            model.addAttribute("errorMessage", "Category not found.");
            return "redirect:/category";
        }
        model.addAttribute("id", c.getCatcode());
        model.addAttribute("desc", c.getCatdesc());
        return "catser";
    }

    @PostMapping("/update-todo")
    @Transactional
    public String edit(@RequestParam String oldId,
                      @RequestParam String catcode,
                      @RequestParam String catdesc,
                      RedirectAttributes ra) {

        if (!oldId.equals(catcode) &&
            categoryRepo.existsById(catcode)) {
            ra.addFlashAttribute("errorMessage", "New category code
already exists.");
            return "redirect:/category";
        }

        int n = categoryRepo.updateCategory(oldId, catcode, catdesc);
        if (n == 0) {
            ra.addFlashAttribute("errorMessage", "No record
updated.");
        } else {
            ra.addFlashAttribute("successMessage", "Category
updated.");
        }
        return "redirect:/category";
    }

    // add imports:
    // import
    org.springframework.web.servlet.mvc.support.RedirectAttributes;

    @GetMapping("/delete-todo")
    public String delete(@RequestParam("id") String id,
                        RedirectAttributes ra) {
        int children = categoryRepo.countItems(id);
        if (children > 0) {
            ra.addFlashAttribute("errorMessage", "Cannot delete:
category has " + children + " item(s). Delete items first.");
            return "redirect:/category";
        }
    }

```

```

    }
    categoryRepo.deleteById(id);
    ra.addFlashAttribute("successMessage", "Category deleted.");
    return "redirect:/category";
}

@GetMapping("/see-todo")
public String items(@RequestParam("id") String id, Model model) {
    var cat = categoryRepo.findWithItems(id).orElse(null); // load
items via relation
    if (cat == null) {
        model.addAttribute("errorMessage", "Category not found.");
        return "redirect:/category";
    }
    if (cat.getItems() == null || cat.getItems().isEmpty()) {
        model.addAttribute("errorMessage", "No items for this
category.");
        return "redirect:/category";
    }

    model.addAttribute("desc", cat.getCatdesc());
    model.addAttribute("itemlist", cat.getItems());
    return "items";
}
}

```

Your program would look like the one below:

Note: The codes above only provide a way to add a category. If you want to see if the item page to show data, you can open MySQL and add data to the items table manually. Make sure that the catcode is existing in the category table. Alternatively, as a practice, you can develop the item form to allow adding of items.

Item Categories

Add

Category deleted.

Code	Description	Actions		
A1	Office	Edit	Delete	Items
B2	Kitchen	Edit	Delete	Items
M2	dddd	Edit	Delete	Items
M3	Medicine	Edit	Delete	Items

Add or Edit Category

Category Code

A1

Category Description

Office

Submit

Cancel

Items for Category: Office

Item Code	Item Description
I001	Desk
I002	Chair

Back